

GENADA: generative time series adversarial attack framework

No Author Given

No Institute Given

Abstract. Deep learning methods are widely used for time series analysis in domains such as healthcare, finance, energy systems, and environmental monitoring. However, deep neural networks remain vulnerable to adversarial attacks, where small input perturbations can cause significant degradation in predictive performance. Commonly used gradient-based adversarial attacks (e.g., iterative first-order methods) are computationally burdensome. They require repeated backpropagation through the victim model to compute input gradients, and many iterative refinement steps to obtain high-quality perturbations under the chosen constraint set. We propose a generative adversarial attack GENADA that learns to produce valid perturbations directly. During training, the generative model learns to produce perturbations that maximize the loss of a frozen target model. We introduce both single-step and iterative generative attack schemes and evaluate them on multiple time series datasets and model architectures. We validate the approach in the time-series domain, which provides a controlled low-dimensional setting for systematic experimentation while still capturing structured, temporally dependent signals. Empirically, we observe that the generative attack attains superior effectiveness to strong baselines (in terms of the task-relevant degradation metrics) with comparable attack quality. The experiments are conducted on 3 datasets with achieved metrics comparable to those for the strongest alternative.

Keywords: First keyword · Second keyword · Another keyword.

1 Introduction

Sequential data arise in a wide range of applications, including healthcare, finance, energy systems, and environmental monitoring [Marusov and Zaytsev, 2023]. One of the central problems in this domain is time series classification, which plays a crucial role in forecasting, anomaly detection, and medical diagnosis [Ismail Fawaz et al., 2019]. Neural networks are among the most effective model families for time series classification tasks [Ismail Fawaz et al., 2019].

Despite their strong predictive performance, neural networks are vulnerable to adversarial attacks: small, often imperceptible perturbations of the input may lead to significant drops in classification accuracy. While adversarial attacks have been extensively studied in computer vision [Akhtar and Mian, 2018] and natural

language processing [Goyal et al., 2023], time series data remain comparatively underexplored. At the same time, sequential signals possess specific structural properties—such as temporal correlations, smoothness constraints, and physical plausibility—that must be taken into account when designing attacks [Sokerin et al., 2025].

Adversarial attacks are commonly categorized into white-box attacks, where gradients of the target model are available, and black-box attacks [Zhang et al., 2022], where such access is absent. Although white-box attacks typically achieve higher attack effectiveness, they require access to the internal model, which is often unrealistic in practice. Moreover, because differentiation of the model tends to be slow, gradient-based attacks can be computationally expensive.

In this work, we propose a generative adversarial attack framework in which a separate neural network is trained to generate perturbations that maximize the target classifier’s loss. Importantly, once trained, the generator can produce adversarial perturbations without computing gradients of the target model at inference time.

Our contributions are as follows:

1. We propose GENADA (GENerative ADversarial Attack), a generative adversarial attack that learns to produce valid perturbations directly. Once trained, the attacker outputs an adversarial perturbation by forward pass, avoiding per-example iterative optimization and eliminating the need for repeated gradient computations at inference time. The proposed attack is straightforward to train with standard learning pipelines and is substantially more efficient to run.
2. We propose a training procedure based on a frozen target model, enabling gradient-free inference.
3. We validate the approach in the time-series domain, which provides a controlled low-dimensional setting for systematic experimentation while still capturing structured, temporally dependent signals. Empirically, we observe that the GENADA attains superior computational efficiency to strong baselines (in terms of the task-relevant degradation metrics) with comparable attack quality. The experiments are conducted on PowerCons, Gun-Point and Strawberry datasets with metrics achieved, compared to metrics for the strongest alternative. We provide an extensive empirical evaluation across multiple datasets and architectures (recurrent, convolutional, and transformer-based models), comparing against strong white-box baselines such as FGSM and iFGSM.
4. We demonstrate that the proposed approach achieves competitive performance while significantly reducing attack generation time.

2 Related works

2.1 Adversarial Attacks

Deep neural networks are known to be susceptible to adversarial perturbations [Szegedy et al.]. Adversarial attacks and methods for generating them

have been extensively studied in recent years across various domains Kurakin et al. [2017], Jia and Liang [2017], Fursov et al. [2020, 2021]. In the literature, adversarial example generation methods are typically categorized according to factors such as access to the target model (white-box vs. black-box), the distortion measure (e.g., ℓ_p -norms), and the principle underlying example search: gradient-based, optimization-based, or generative-based Akhtar and Mian [2018]. In general, constructing a strong adversarial example corresponds to maximizing the loss function of the target model within an ϵ -ball around the original input.

Classical white-box methods rely on explicit gradients of the classification loss $\mathcal{L}_{\text{cls}}$ with respect to the input $\mathbf{X}$. The Fast Gradient Sign Method (FGSM) and its iterative variants (iFGSM), as well as Projected Gradient Descent (PGD), represent standard first-order attack baselines. These classical methods establish reference benchmarks for evaluating new algorithms.

2.2 Time series classification

Time series classification (TSC) is a central task in time series analysis Yang and Wu [2006] and appears in many practical settings, including finance and healthcare.

A large part of recent progress has been enabled by standardized benchmark suites, most notably the UCR archive Dau et al. [2019] and the multivariate UEA archive Bagnall et al. [2018]. They cover a wide range of domains (for example, UCR includes 100+ datasets such as activity recognition, ECG, and sensor-based problems), which makes comparisons between methods more reliable. Following this benchmarking landscape, we selected three diverse datasets for univariate TSC.

From a modeling perspective, modern TSC is often associated with deep learning, yet the field does not have a universal winner: different architectures excel under different data characteristics and constraints. Convolutional models (e.g., Residual CNN, ResCNN Zou et al. [2019]) leverage ResNet-style skip connections He et al. [2016] to stabilize optimization and capture multi-scale patterns. Sequence-based approaches such as RNNs and LSTMs Hochreiter and Schmidhuber [1997] remain relevant for long-range temporal structure, and hybrids like RNNAttention Oguiza [2023] add attention mechanisms on top of recurrence. Transformer-based methods have recently become popular; PatchTST Nie et al. [2023] processes time series as patches and uses self-attention to model long dependencies efficiently. Another emerging direction is state space modeling: S4 Gu et al. [2022] uses a structured parameterization of state matrices to scale to long sequences while offering connections to both recurrent and convolutional formulations.

Despite these deep learning advances Mohammadi Foumani et al. [2024], classical approaches formed the backbone of TSC for many years. Techniques based on Dynamic Time Warping with nearest neighbors Keogh and Ratanamahatana [2005], as well as strong feature-based ensembles such as COTE Bagnall et al. [2015] and Weasel Schäfer and Leser [2017], were competitive SOTA methods throughout the last decade. Hybrid ensembles continue to be strong:

HIVE-COTE 2.0 (HC2) Middlehurst et al. [2021] combines four complementary components (TDE, DrCIF, The Arsenal, and STC) and remains among the most effective non-deep baselines Middlehurst et al. [2024].

2.3 Adversarial Attacks for Sequential Data

Adversarial attacks are extensively studied in computer vision—where carefully designed, small perturbations can consistently fool image classifiers—but they have received noticeably less attention in time series classification (TSC). This gap is increasingly important given the rapid adoption of deep learning for TSC and the still limited understanding of adversarial risks in sequential settings Ding et al. [2023].

A common starting point is gradient-based white-box attacks, which expose model vulnerabilities across domains Akhtar and Mian [2018], Tramèr et al. [2018]. In the time-series context, Fawaz Fawaz et al. [2019] shows that straight-forward iterative methods such as iFGSM and BIM can be effective, motivating their use as baseline attacks.

However, sequential data introduce constraints that are less prominent in images or text: temporal dependence, smoothness, and application-specific feasibility requirements. To address these issues, recent work Forestier and coauthors [2022] adapts iterative attacks to time series and evaluates imperceptibility with criteria beyond simple norm bounds, including smoothness and bounded variation.

Beyond direct gradient-based optimization, several methods are designed specifically for time series. For example, adversarial transformation networks learn a neural generator that produces adversarial examples Karim et al. [2020], and subsequent work extends this idea to multivariate sequences, demonstrating broader applicability Harford et al. [2020].

Overall, while adapting vision-style attacks such as FGSM/BIM to time series often works in practice, two research directions remain central: making perturbations less perceptible under temporal constraints and developing attacks that explicitly exploit the structure of sequential data.

2.4 Surrogate Models and Transferability

When gradients are unavailable, adversarial examples can be constructed using surrogate models. This approach relies on the transferability phenomenon Papernot et al. [2017]: models trained for the same task can exhibit similar gradient directions for increasing the loss function. Consequently, adversarial examples crafted for one model often transfer to other models trained to solve related tasks.

The effectiveness of such attacks depends on the alignment between the loss landscapes of surrogate and target models. Accordingly, a surrogate model should be trained on the same training set and, ideally, use a similar architecture, hyperparameters, and other design choices.

Our approach differs in that we train a dedicated generative model to directly produce transferable perturbations for time series data.

2.5 RESEARCH GAP

3 Methodology

3.1 Problem statement

Adversarial attack

$$\mathcal{L}_{\text{cls}}(\mathbf{X} + \boldsymbol{\delta}, Y) \rightarrow \max, \|\boldsymbol{\delta}\| < \epsilon$$

Time Series Classification. Let $f(\mathbf{X})$ denote a target classifier operating on multivariate time series $\mathbf{X}$. The dataset is given by $\mathcal{D} = \{(\mathbf{X}_i, y_i)\}_{i=1}^n$, where $\mathbf{X}_i \in \mathbb{R}^d$ is an input sequence and $y_i \in \mathcal{Y}$ is the corresponding class label. The model f is trained using cross-entropy loss $\mathcal{L}_{\text{cls}}(f(\mathbf{X}), y)$. **Adversarial Attacks.** An adversarial attack constructs a perturbed example $\mathbf{X}'$ by adding perturbations $\boldsymbol{\delta}$ to the original data object $\mathbf{X}' = \mathbf{X} + \boldsymbol{\delta}$, such that $f(\mathbf{X}) \neq f(\mathbf{X}')$, while some norm like l_2 or l_∞ for the perturbation $\|\boldsymbol{\delta}\|$ is upper-bounded. **Problems with gradient attacks**

Gradient-based attacks such as iFGSM or PGD are inefficient because they perform computationally expensive backpropagation T times for T iterations. This cost grows even more quickly when the model is large, when the attack must satisfy structured constraints, or when many candidate perturbations must be generated to achieve desired quality [TODO: references needed].

Zero-order generation of adversarial example

$$\mathcal{L}_{\text{cls}}(\mathbf{X} + \boldsymbol{\delta}_{\mathbf{X}}, Y) \rightarrow \max, \|\boldsymbol{\delta}_{\mathbf{X}}\| < \epsilon$$

In contrast, a generative attack after training is expected to produce strong attacks in a single forward pass, making inference much faster and more scalable. Moreover, fine-tuning a generative model enables the incorporation of specific properties, such as learning perturbations that better respect domain structure, for example, temporal smoothness in time series.

3.2 Gradient-Based Baselines

As a baseline, let's consider classical attacks based on explicit gradients of the loss function of the target classifier.

In the remaining of the paper we use the following notation:

$$\text{Clip}_{(\mathbf{X}, \epsilon)}(\mathbf{X}') := \min\{\max\{\mathbf{X}', \mathbf{X} - \epsilon\}, \mathbf{X} + \epsilon\}$$

Fast Gradient Sign Method (FGSM) Given an constraint $\|\boldsymbol{\delta}\|_\infty \leq \epsilon$, FGSM perturbs the input by taking a single step in the direction of the gradient sign:

$$\mathbf{X}' = \mathbf{X} + \epsilon \text{sign}(\nabla_{\mathbf{X}} \mathcal{L}_{\text{cls}}(\mathbf{X}, y)).$$

Iterative FGSM (iFGSM) A multi-step version refines the perturbation over T iterations with step size $\epsilon/T \leq \alpha \leq \epsilon$, keeping the cumulative distortion within an ℓ_∞ -ball of radius ϵ :

$$\mathbf{X}^0 = \mathbf{X}, \quad \mathbf{X}^{(t+1)} = \text{Clip}_{(\mathbf{X}, \epsilon)} \left(\mathbf{X}^{(t)} + \alpha \text{sign} \left(\nabla_{\mathbf{X}} \mathcal{L}_{\text{cls}}(\mathbf{X}^{(t)}, y) \right) \right).$$

3.3 GENADA

Generative attacks (GENADA) Our goal is to develop a generative adversarial attack h that manipulates the target model’s predictions. To achieve this, our method employs an additional trainable attacking model G to generate adversarial perturbations. More formally, $\delta = h(\mathbf{X}) = a(G(\mathbf{X}))$, where a is a post-processing of the model outputs. In Section 3.3, we provide a more detailed description of the generative attack and training procedures for generative models.

GENADA The main idea of a generative adversarial attack is to perturbations δ not by using gradients of the target model, access to which may be limited, but by using a separate attacking model G . In this work, we propose two types of generative adversarial attacks: the generative adversarial attack h_G and the iterative generative adversarial attack $h_{G_{\text{iter}}}$.

In the h_G attack, the attacking model takes the features as input and generates an $\tilde{\delta}$ perturbation vector of the same dimension as the original data. However, such a vector cannot be directly used as a perturbation, since its range is unlimited. To preserve the amplitude of the attack, we use the regularization mechanism $a(\tilde{\delta})$: the output of the model $G(\mathbf{X})$ is placed in a non-linear function that limits the noise range. In our case, at the learning stage, we apply the hyperbolic tangent $\tanh(\cdot)$, which guarantees ℓ_∞ -boundedness. Then multiply the noise by the ϵ attack force, as in FGSM.

We use a generator network G that produces perturbations Baluja and Fischer [2018]:

$$\delta = \epsilon \tanh(G(\mathbf{X})).$$

The perturbed input becomes:

$$\mathbf{X}' = \mathbf{X} + \delta.$$

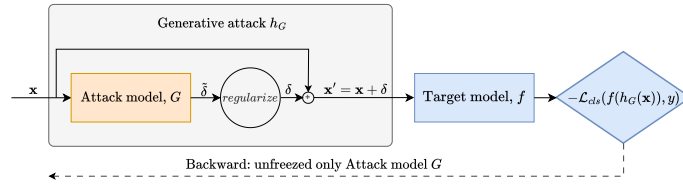
The generator is trained to maximize the classification loss of a frozen target model:

$$\max_{\theta_G} \mathcal{L}_{\text{cls}}(f(\mathbf{X} + \epsilon \tanh(G_{\theta_G}(\mathbf{X})), y)).$$

The backpropagation algorithm and gradient *ascent* with respect to the parameters of the generator model G were used.

A detailed attack scheme is shown in Figure 1.

During inference, when evaluating attack metrics with a trained generator G , the regularization function a was replaced with $\text{sign}(\cdot)$. This ensures that the

**Fig. 1.** GENADA pipeline

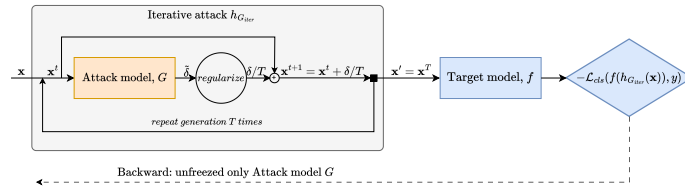
ℓ_∞ norm of the attack δ is scaled exactly to ϵ , whereas $\tanh(\cdot)$ merely constrains the attack within an ℓ_∞ bound. During training, we opted for $\tanh(\cdot)$ because it is smoother than $\text{sign}(\cdot)$, which facilitates more effective gradient flow.

3.4 Iterative GENADA

Analogous to iFGSM, we propose an iterative version in which perturbations are applied over T steps. The attack noise generation step at each iteration corresponds to δ/T so that the resulting attack conforms to δ , while maintaining the total ℓ_∞ constraint.

Here, similarly, during training we used $\tanh(\cdot)$ as the regularization function $a(\cdot)$ to improve gradient flow, while at inference with the trained generator we applied $\text{sign}(\cdot)$ to increase the attack strength in terms of the ℓ_∞ norm.

The principle of an iterative attack is to attack data sequentially T times, see Figure 2 and Algorithm 1. Similar to an iterative version of FGSM, we balance the performance drop after the attack and the computational cost by selecting T .

**Fig. 2.** Iterative GENADA pipeline

3.5 Model Architectures

TODO: why is this part not in the experiments? Did you propose anything specific here?

We consider three architectures for both classifiers and generators:

Algorithm 1 Iterative GENADA

Require: Input time series $\mathbf{X}$, label y , frozen classifier f , generator G , attack budget ϵ , number of steps T

Ensure: $\mathbf{X}^{(T)}$ ▷ adversarial example

- 1: $\mathbf{X}^{(0)} \leftarrow \mathbf{X}$ ▷ initialize the iterative process from the original input
- 2: **for** $t = 0, 1, \dots, T - 1$ **do** ▷ perform T refinement steps
- 3: $\tilde{\delta}^{(t)} \leftarrow G(\mathbf{X}^{(t)})$ ▷ the generator predicts a perturbation
- 4: $\delta^{(t)} \leftarrow \frac{\epsilon}{T} a(\tilde{\delta}^{(t)})$ ▷ scale the perturbation to control the total strength
- 5: $\mathbf{X}^{(t+1)} \leftarrow \mathbf{X}^{(t)} + \delta^{(t)}$ ▷ update the adversarial example by adding the generated perturbation
- 6: $\mathbf{X}^{(t+1)} \leftarrow \text{Clip}_{(\mathbf{X}, \epsilon)}(\mathbf{X}^{(t+1)})$ ▷ project $\mathbf{X}^{(t+1)}$ into the allowed ℓ_∞ -ball around $\mathbf{X}$
- 7: **end for**
- 8: **return** $\mathbf{X}^{(T)}$ ▷ output the final adversarial example

- **LSTM-based models** We adopt a Long Short-Term Memory (LSTM) network as the baseline. The target model is trained by passing the hidden state of the last time step through a fully connected layer followed by **sigmoid/softmax** activation to obtain class probabilities. Adversarial perturbations are generated by a separate LSTM generator consisting of two consecutive LSTM blocks. An element-wise nonlinearity $\alpha(\cdot)$ — either identity, ReLU, or **tanh** — is applied between the blocks:

$$\mathbf{z} = \text{LSTM}_{\text{in}}(\mathbf{X}) \quad \hat{\mathbf{z}} = \alpha(\mathbf{z}) \quad \mathbf{u} = \text{LSTM}_{\text{out}}(\hat{\mathbf{z}}) \quad (1)$$

A linear projection then maps $\mathbf{u}$ back to the input dimensionality, producing a perturbation sequence $\tilde{\delta}$ of the same length and dimension as the original time series.

- **Residual CNN (ResCNN)** For the second model, we employ a residual convolutional network (ResCNN) that processes time series through a sequence of convolutional layers with skip connections. The architecture operates directly on the input without recurrent components. For perturbation generation, we adapt this design into a lightweight Mini-ResCNN generator. It comprises two shallow ResCNN modules arranged similarly to the LSTM-based generator described above, forming the function $\text{ResCNN}(\cdot)$.
- **PatchTST transformer-based architecture** PatchTST is a Transformer-based model for sequential data that employs a patching mechanism to obtain embeddings of input subsequences. The input $\mathbf{X}$ is divided into overlapping patches and processed by a Transformer encoder. While originally designed for time series classification, it can also be adapted for perturbation generation. Our adversarial generator follows the same scheme as previous models but uses a *single* PatchTST encoder with a linear head. First, a global latent vector is extracted and then repeated along the time dimension:

$$\mathbf{z} = \text{PatchTST}_{\text{enc}}(\mathbf{x}), \quad \mathbf{u} = \alpha(\hat{\mathbf{z}}) \quad (2)$$

Generators follow the same architectural backbone as the corresponding classifier, with a final linear projection producing perturbations of the same dimensionality as the input.

Problem Setup Let $x \in \mathbb{R}^d$ be an input and $y \in \mathcal{Y}$ its label. Let $C : \mathbb{R}^d \rightarrow \mathbb{R}^K$ be a fixed classifier and $\ell : \mathbb{R}^K \times \mathcal{Y} \rightarrow \mathbb{R}$ a loss function.

$$f_x(\delta) := \ell(C(x + \delta), y), \quad \|\delta\| \leq \varepsilon.$$

We consider the maximization problem

$$f_x(\delta) \rightarrow \max \quad \text{s.t. } \|\delta\| \leq \varepsilon.$$

$$\delta_x^* = \arg \max_{\|\delta\| \leq \varepsilon} f_x(\delta).$$

Let the generator be a parametric mapping

$$G_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d,$$

and we train it by **maximizing** the objective

$$J(\theta) := \mathbb{E}_{x \sim \mathcal{D}} f_x(G_\theta(x)).$$

Assumptions TODO: why do we consider these assumptions, how do they relate to assumptions typically used in the literature?

We consider three assumptions:

A1 L -smoothness of $f_x(\delta)$ with respect to δ :

$$\forall x, \delta_1, \delta_2 : \|\delta_i\| \leq \varepsilon \Rightarrow \|\nabla f_x(\delta_1) - \nabla f_x(\delta_2)\| \leq L\|\delta_1 - \delta_2\|.$$

A2 Uniform approximation capacity of the generator:

$$\forall \eta > 0 \exists \theta^* : \sup_x \|G_{\theta^*}(x) - \delta_x^*\| \leq \eta.$$

A3 Polyak–Łojasiewicz (PL) condition for $J(\theta)$:

$$\exists \mu_\theta > 0 : \forall \theta \Rightarrow \frac{1}{2} \|\nabla_\theta J(\theta)\|^2 \geq \mu_\theta (J(\theta^*) - J(\theta)).$$

Theorem TODO: what the theorem is about? Why is it important? What gap does it fill?

Theorem 1. Suppose the assumptions [A1]–[A3] hold, and let θ_k satisfy

$$\|\nabla_\theta J(\theta_k)\| \leq \varepsilon_{\text{grad}}.$$

Then

$$\mathbb{E}_x [f_x(\delta_x^*) - f_x(G_{\theta_k}(x))] \leq \frac{L}{2} \eta^2 + \frac{\varepsilon_{\text{grad}}^2}{2\mu_\theta}.$$

Proof.

I. Approximation error contribution. By the approximation capacity of the generator, for any $\eta > 0$ there exists θ^* such that

$$\|G_{\theta^*}(x) - \delta_x^*\| \leq \eta \quad \forall x.$$

By L -smoothness of f_x , it admits the quadratic upper bound

$$\forall \delta_1, \delta_2 \Rightarrow f_x(\delta_1) \leq f_x(\delta_2) + \langle \nabla f_x(\delta_2), \delta_1 - \delta_2 \rangle + \frac{L}{2} \|\delta_1 - \delta_2\|^2.$$

Applying this inequality with $\delta_1 = G_{\theta^*}(x)$ and $\delta_2 = \delta_x^*$, we obtain

$$f_x(G_{\theta^*}(x)) \leq f_x(\delta_x^*) + \langle \nabla f_x(\delta_x^*), G_{\theta^*}(x) - \delta_x^* \rangle + \frac{L}{2} \eta^2.$$

The first-order optimality condition for δ_x^* in the constrained maximization problem over the ε -ball yields

$$\forall \delta' : \|\delta'\| \leq \varepsilon \Rightarrow \langle \nabla f_x(\delta_x^*), \delta' - \delta_x^* \rangle \leq 0.$$

Taking $\delta' = G_{\theta^*}(x)$ gives

$$\langle \nabla f_x(\delta_x^*), G_{\theta^*}(x) - \delta_x^* \rangle \leq 0,$$

and therefore

$$f_x(\delta_x^*) - f_x(G_{\theta^*}(x)) \leq \frac{L}{2} \eta^2.$$

Taking expectation over x :

$$J(\theta^*) \geq \mathbb{E}_x f_x(\delta_x^*) - \frac{L}{2} \eta^2.$$

II. Optimization error contribution. From the PL condition for $J(\theta)$ and $\|\nabla_\theta J(\theta_k)\| \leq \varepsilon_{\text{grad}}$,

$$\frac{1}{2} \varepsilon_{\text{grad}}^2 \geq \mu_\theta (J(\theta^*) - J(\theta_k)),$$

hence

$$J(\theta^*) - J(\theta_k) \leq \frac{\varepsilon_{\text{grad}}^2}{2\mu_\theta}.$$

III. Final bound.

$$\mathbb{E}_x f_x(\delta_x^*) - J(\theta_k) = (\mathbb{E}_x f_x(\delta_x^*) - J(\theta^*)) + (J(\theta^*) - J(\theta_k)).$$

Using the bounds from Steps I and II,

$$\mathbb{E}_x f_x(\delta_x^*) - J(\theta_k) \leq \frac{L}{2} \eta^2 + \frac{\varepsilon_{\text{grad}}^2}{2\mu_\theta}.$$

4 Experiments

4.1 Datasets

We evaluate on three datasets from the UCR Time Series Archive [Dau et al., 2019]:

- *PowerCons* ($L = 144$, binary classification). The PowerCons dataset contains daily electricity consumption profiles for a single household over one year. The data is split into a training set (180 profiles) and a test set (180 profiles). Each profile represents a single day’s consumption, consisting of 144 data points recorded at 10-minute intervals. The profiles are labeled into two classes based on the season: warm (class 1, April-September) and cold (class 2, October-March). Consumption patterns vary significantly within each season.
- *Strawberry* ($L = 235$, binary classification). The Strawberry dataset contains food spectrographs obtained via FTIR-ATR spectroscopy, used to distinguish authentic strawberry purees from adulterated samples (including non-strawberry purees). Each time series is of length 235. The dataset is split into 613 training and 370 test instances, with no missing values.
- *GunPoint* ($L = 150$, binary classification). The GunPoint dataset contains motion tracking data of actors performing two actions: drawing and pointing a gun (class Gun) and pointing with a finger (class Point). Each time series of length 150 tracks the x-axis centroid of the actor’s right hand. The dataset is split into 50 training and 150 test instances, with no missing values.

4.2 Evaluation Metrics

In our experiments, we used three evaluation metrics: fooling rate, effectiveness Sokerin et al. [2025], and target accuracy. **Fooling Rate (FR)** measures the proportion of instances that change their prediction after the attack:

$$FR = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(f(\mathbf{X}_i) \neq f(\mathbf{X}'_i)).$$

where N is the sample size, $\mathbf{X}_i$ is the original instance, $\mathbf{X}'_i$ is the adversarial instance, and $f(\cdot)$ is the target model.

Attack Effectiveness (E): characterizes the degradation of the target model’s performance. It is defined as the difference between 1 and the F_1 -score computed on the perturbed examples $\mathbf{X}'$:

$$E = 1 - F_1(y^{\text{true}}, f(\mathbf{X}')).$$

where y^{true} denotes the ground truth class labels.

Target Accuracy (TA): measures the proportion of instances that remain correctly classified after the attack. It is essentially the standard *accuracy* metric

computed on the perturbed examples:

$$TA = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(f(\mathbf{x}'_i) = y_i).$$

where y_i^{true} is the ground truth label for the i -th instance.

4.3 Experimental setup

All experiments were run on a single NVIDIA RTX A5000 GPU. Since the UCR datasets are relatively small, training each classifier and generator required only modest computational effort.

In our experiments, we used the following hyperparameter settings: The datasets were normalized beforehand. For all types of adversarial attacks, perturbation size ϵ values ranging from 0.1 to 0.5 were used. To train both classification and attack generation models, learning rate was set to either 0.001 or 0.0001. Adam and AdamW acted as optimizers with a batch size of 64.

More specifically:

- **Target models:** The maximum number of epochs was set to 100. Training time varied from approximately 30 seconds to 3 minutes, depending on the dataset and model.
- **Baseline attacks:** FGSM adversarial examples were generated using a single gradient step. The iFGSM attack was run for approximately $T = 10$ iterations.
- **GENADA:** The number of steps for iterative GENADA was no more than 20. In very few cases, 50 iterations were set. The number of training epochs did not exceed 200. It was mainly around 100. Training for simple datasets required fewer epochs and took about 1-2 minutes. For the more complex Strawberry dataset, the maximum training time was about 5 minutes.

A bottleneck was hyperparameter selection rather than training. We spent substantial time tuning learning rates, dropout, attack-iteration schedules, and model-specific settings to achieve stable learning and reliable detection of subtle perturbations. This tuning process contributed substantially to the overall experimental runtime.

The source code is available in the repository: [anonymized github-repository](#).

4.4 Main Results

On the PowerCons dataset, the iterative GENADA achieves effectiveness comparable to iFGSM. In several settings, the proposed approach significantly outperforms single-step gradient variants. Moreover, GENADA demonstrates substantial speed improvements—between $10\times$ and $50\times$ faster at inference time—since no gradient computation is required.

Transferability experiments show that attacks remain effective even when generator and target architectures differ, confirming applicability in black-box scenarios.

On the Strawberry dataset, performance remains below strong white-box baselines, highlighting directions for future work.

5 Conclusion

We proposed a GENADA framework for time series classification that does not require gradient access to the target model during inference. Both single-step and iterative variants achieve competitive performance compared to classical white-box attacks while providing significant computational advantages.

The results demonstrate that generative models constitute a promising alternative to gradient-based attacks, especially in restricted-access scenarios. Future work includes improving generator architectures, enhancing perturbation imperceptibility, and extending evaluation to larger and more diverse datasets.

References

- Naveed Akhtar and Ajmal Mian. Threat of adversarial attacks on deep learning in computer vision: a survey. *IEEE Access*, 6:14410–14430, 2018.
- Anthony Bagnall, Jason Lines, Jon Hills, and Aaron Bostrom. Time-series classification with cote: the collective of transformation-based ensembles. *IEEE Trans. Knowl. Data Eng.*, 27(9):2522–2535, 2015.
- Anthony Bagnall, Hoang Anh Dau, Jason Lines, Michael Flynn, James Large, Aaron Bostrom, Paul Southam, and Eamonn Keogh. The UEA multivariate time series classification archive, 2018. *arXiv preprint arXiv:1811.00075*, 2018.
- Shumeet Baluja and Ian Fischer. Adversarial transformation networks: Learning to generate adversarial examples. In *AAAI*, 2018.
- Hoang Anh Dau, Anthony Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn Keogh. The ucr time series archive. *IEEE/CAA Journal of Automatica Sinica*, 6(6):1293–1305, November 2019. <https://doi.org/10.1109/JAS.2019.1911747>.
- Daizong Ding, Mi Zhang, Fuli Feng, Yuanmin Huang, Erling Jiang, and Min Yang. Black-box adversarial attack on time series classification. *Proc. AAAI Conf. Artif. Intell.*, 37(6):7358–7368, Jun. 2023. <https://doi.org/10.1609/aaai.v37i6.25896>. URL <https://ojs.aaai.org/index.php/AAAI/article/view/25896>.
- Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. Adversarial attacks on deep neural networks for time series classification. In *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, pages 1–8. IEEE, 2019.
- Germain Forestier and coauthors. Smooth perturbations for time series adversarial attacks. 2022. PKDD / JDSA.

- Ivan Furosov, Alexey Zaytsev, Nikita Kluchnikov, Andrey Kravchenko, and Evgeny Burnaev. Differentiable language model adversarial attacks on categorical sequence classifiers. *arXiv preprint arXiv:2006.11078*, 2020.
- Ivan Furosov, Alexey Zaytsev, Pavel Burnyshev, Ekaterina Dmitrieva, Nikita Klyuchnikov, Andrey Kravchenko, and Evgeny Burnaev. A differentiable language model adversarial attack on text classifiers. *IEEE Access*, 9:17966–17984, 2021.
- Shreya Goyal et al. A survey of adversarial defenses and robustness in NLP. *ACM Comput. Surv.*, 55(14s):1–39, 2023.
- Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, pages 1–32, 2022.
- Samuel Harford, Fazle Karim, and Houshang Darabi. Adversarial attacks on multivariate time series. *arXiv preprint arXiv:2004.00410*, 2020.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proc. IEEE Conf. Comput. Vision Pattern Recognit.*, pages 770–778, 2016.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, November 1997. ISSN 0899-7667. <https://doi.org/10.1162/neco.1997.9.8.1735>. URL <https://doi.org/10.1162/neco.1997.9.8.1735>.
- Hassan Ismail Fawaz et al. Deep learning for time series classification: a review. *Data mining and knowledge discovery*, 33(4):917–963, 2019.
- Robin Jia and Percy Liang. Adversarial examples for evaluating reading comprehension systems. In *EMNLP*, pages 2021–2031, Copenhagen, Denmark, September 2017. Association for Computational Linguistics.
- Fazle Karim, Somshubra Majumdar, and Houshang Darabi. Adversarial attacks on time series. *IEEE Trans. Pattern Anal. Mach. Intell.*, 43(10):3309–3320, 2020.
- Eamonn Keogh and Chotirat Ann Ratanamahatana. Exact indexing of dynamic time warping. *Knowl. Inf. Syst.*, 7:358–386, 2005.
- Alexey Kurakin, Ian Goodfellow, and Samy Bengio. Adversarial examples in the physical world. In *ICLR*, 2017.
- Alexander E Marusov and Alexey Zaytsev. Noncontrastive representation learning for intervals from well logs. *IEEE Geosci. Remote Sens. Lett.*, 20:1–5, 2023.
- Matthew Middlehurst, James Large, Michael Flynn, Jason Lines, Aaron Bostrom, and Anthony Bagnall. Hive-cote 2.0: a new meta ensemble for time series classification. *Mach. Learn.*, 110(11-12):3211–3243, 2021.
- Matthew Middlehurst, Patrick Schäfer, and Anthony Bagnall. Correction: Bake off redux: a review and experimental evaluation of recent time series classification algorithms. *Data Mining and Knowledge Discovery*, 38(6):4236–4237, 2024. <https://doi.org/10.1007/s10618-024-01040-z>.
- Navid Mohammadi Foumani, Lynn Miller, Chang Wei Tan, Geoffrey I Webb, Germain Forestier, and Mahsa Salehi. Deep learning for time series classification and extrinsic regression: A current survey. *ACM Comput. Surv.*, 56(9): 1–45, 2024.

- Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, pages 1–24, 2023.
- Ignacio Oguiza. tsai - a state-of-the-art deep learning library for time series and sequential data. Github, 2023. URL <https://github.com/timeseriesAI/tsai>.
- Nicolas Papernot et al. Practical black-box attacks against machine learning. In *ASIACCS*, pages 506–519, 2017.
- Patrick Schäfer and Ulf Leser. Fast and accurate time series classification with weasel. In *Proc. ACM Conf. Inf. Knowl. Manage.*, pages 637–646, 2017.
- Petr Sokerin, Dmitry Anikin, Sofia Krehova, and Alexey Zaytsev. Concealed adversarial attacks on neural networks for sequential data. *arXiv preprint arXiv:2502.20948*, 2025.
- Christian Szegedy et al. Intriguing properties of neural networks.
- Florian Tramèr, Alexey Kurakin, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. Ensemble adversarial training: Attacks and defenses. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, pages 1–22, 2018.
- Qiang Yang and Xindong Wu. 10 challenging problems in data mining research. *Int. J. Inf. Technol. Decis. Making*, 5(4):597–604, 2006. <https://doi.org/10.1142/S0219622006002258>. URL <https://doi.org/10.1142/S0219622006002258>.
- Chaoning Zhang et al. Investigating top-k white-box and transferable black-box attack. In *CVPR*, pages 15085–15094, 2022.
- Xiaowu Zou, Zidong Wang, Qi Li, Weiguo Sheng, and coauthors. Integration of residual network and convolutional neural network along with various activation functions and global pooling for time series classification. *Neurocomputing*, 367:39–45, 2019.